

ASSESSMENT TOOL – 1

CASE STUDY

CASE STUDY SCENARIO

A multi-specialty hospital is planning to modernize its Hospital Management System (HMS) to efficiently manage different types of patients such as Inpatients, Outpatients, and Emergency Patients.

The current system has several issues:

- ✓ Redundant data storage for patient details
- ✓ Lack of flexibility in handling different billing mechanisms
- ✓ Difficulty in extending the system for new patient categories
- ✓ Inefficient handling of patient records in memory

To improve system performance and scalability, the hospital wants to redesign the system using advanced Object-Oriented Programming concepts in C++.

he system using advanced Object-Oriented Programming concepts in C++.

Case Study Task

You are required to analyze, design, implement, and evaluate a C++ application using:

- ✓ Inheritance (types of inheritance)
- ✓ Virtual base classes
- ✓ Abstract classes
- ✓ Constructors in derived classes
- ✓ Member classes (composition)
- ✓ Pointers and pointers to objects
- ✓ this pointer
- ✓ Pointer to derived classes
- ✓ Runtime polymorphism

Part A:

1. Identify the limitations of the existing system.
2. Explain how OOP principles can solve these issues.
3. List the functional requirements of the improved system.

1. Limitations of Existing System

The existing Hospital Management System has these problems:

1. Redundant data storage
Same patient details are stored again and again for different patient types.
2. Poor billing flexibility
Inpatient, outpatient, emergency, and critical patients have different billing methods, but the system cannot handle them easily.
3. Difficult to add new patient categories
Adding a new patient type requires major changes in the existing code.
4. Inefficient memory handling
Patient records are not managed properly in memory, causing wastage.
5. Low maintainability
Code becomes lengthy, repeated, and difficult to update.

2. How OOP Solves These Issues

OOP concepts help in the following ways:

1. Inheritance avoids repeated patient details by keeping common data in one base class.
2. Polymorphism allows different patient types to calculate bills differently using the same function name.
3. Abstract class gives a common structure for all patient categories.
4. Composition allows patient details to include doctor details using a separate Doctor class.
5. Pointers and dynamic memory help to create patient objects at runtime.
6. Virtual base class solves ambiguity in multiple inheritance.

3. Functional Requirements

The improved HMS should:

1. Store patient details.
2. Store assigned doctor details.
3. Handle different patient types:
 - InPatient
 - OutPatient
 - EmergencyPatient
 - CriticalPatient
4. Calculate bills based on patient type.
5. Use runtime polymorphism.
6. Allow dynamic creation and deletion of patient records.

7. Provide menu-driven interaction.
8. Support future patient categories easily.

Part B:

1. Design a class hierarchy:

- o Abstract base class: Patient
- o Derived classes:

InPatient

OutPatient

EmergencyPatient

2. Extend using multiple inheritance:

- o Create CriticalPatient (inherits from InPatient and EmergencyPatient)

3. Use:

- o Virtual base classes to resolve ambiguity
- o A member class (Doctor) inside Patient

Class Hierarchy Design

Doctor

|

| composition

|

Patient ← Abstract Base Class

|

|-- InPatient

|

|-- OutPatient

|

|-- EmergencyPatient

CriticalPatient

inherits from InPatient and EmergencyPatient

Explanation

- Patient is an abstract base class.
- Doctor is a member class inside Patient.
- InPatient, OutPatient, and EmergencyPatient inherit from Patient.
- CriticalPatient inherits from both InPatient and EmergencyPatient.
- virtual inheritance is used to avoid duplicate copies of Patient.

Part C:

1. Compare:

- o Inheritance vs Composition in patient management
- o Static vs Dynamic binding

2. Analyze:

- o The role of virtual base classes in diamond problem
- o Benefits of pointers in dynamic patient handling

3. Justify:

- o Your design approach
- o How it improves scalability and maintainability

1. Inheritance vs Composition

Topic	Inheritance	Composition
Relationship	Inheritance means is-a relationship. Example: InPatient is a Patient.	Composition means has-a relationship. Example: Patient has a Doctor.
Use in HMS	Used to create different patient types like InPatient, OutPatient, and EmergencyPatient.	Used to include Doctor details inside Patient class.
Code Reuse	Reuses common patient details from the base class.	Reuses separate class objects inside another class.
Flexibility	Less flexible if hierarchy becomes large.	More flexible because one class can contain many objects.
Example	class InPatient : public Patient	class Patient { Doctor d; };

2 . Static Binding vs Dynamic Binding

Static Binding	Dynamic Binding
Decided at compile time	Decided at runtime
Faster	More flexible
Function overloading example	Function overriding example
No virtual function needed	Uses virtual function

3. Role of Virtual Base Class

In multiple inheritance, CriticalPatient inherits from both InPatient and EmergencyPatient.

Both these classes inherit from Patient.

So without virtual base class, two copies of Patient will be created.

This is called the **diamond problem**.

Using:

```
class InPatient : virtual public Patient
```

4. Benefits of Pointers

Pointers help to:

1. Create objects dynamically using new.
2. Store different patient objects using one base class pointer.
3. Achieve runtime polymorphism.
4. Save memory by deleting objects using delete.

5. Design Justification

This design is better because:

5. Common patient details are stored only once.
6. Billing logic is separated for each patient type.
7. New patient types can be added easily.
8. Code is reusable and maintainable.
9. Runtime polymorphism improves flexibility.

Part D:

Develop a C++ program that:

1. Uses:

o Abstract class with pure virtual function calculateBill()

- o Function overriding (polymorphism)

2. Implements billing logic:

- o InPatient → Room charge + Treatment cost

- o OutPatient → Consultation fee

- o Emergency → Additional emergency charges

3. Demonstrates:

- o Pointer to base class (Patient*)

- o Dynamic allocation (new, delete)

- o Use of this pointer

4. Includes:

- o Menu-driven program using control structures

THE CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
//===== MEMBER CLASS =====
```

```
class Doctor
```

```
{
```

```
private:
```

```
    string doctorName;
```

```
public:
```

```
    Doctor()
```

```
{
```

```
    doctorName = "Not Assigned";
```

```
}
```

```

    Doctor(string dname)

    {

        doctorName = dname;

    }

    void displayDoctor()

    {

        cout << "Doctor : " << doctorName << endl;

    }

};

//===== ABSTRACT BASE CLASS =====

class Patient

{

protected:

    string patientName;

    int patientId;

    Doctor doctor; // Composition

public:

    Patient()

    {

        patientName = "";

        patientId = 0;

    }

    Patient(string name,int id,string dname)

        : doctor(dname)

```

```

{
    // this pointer

    this->patientName = name;

    this->patientId = id;
}

virtual void calculateBill() = 0;

void displayPatient()
{
    cout << "\nPatient Name : " << patientName << endl;

    cout << "Patient ID  : " << patientId << endl;

    doctor.displayDoctor();
}

virtual ~Patient() {}
};

//===== INPATIENT =====

class InPatient : virtual public Patient
{
protected:

    float roomCharge;

    float treatmentCost;

public:

    InPatient() {}

    InPatient(string name,int id,string dname,

        float room,float treatment)

        : Patient(name,id,dname)

```



```

    {
        roomCharge = room;
        treatmentCost = treatment;
    }

    void calculateBill()
    {
        displayPatient();

        cout << "Bill = "

            << roomCharge + treatmentCost

            << endl;
    }
};

//===== OUTPATIENT =====

class OutPatient : virtual public Patient
{
private:
    float consultationFee;

public:
    OutPatient(string name,int id,string dname,
        float fee)
        : Patient(name,id,dname)
    {
        consultationFee = fee;
    }

    void calculateBill()

```

```

    {

        displayPatient();

        cout << "Bill = "

            << consultationFee

            << endl;

    }

};

//===== EMERGENCY PATIENT =====

class EmergencyPatient : virtual public Patient

{

protected:

    float emergencyCharge;

public:

    EmergencyPatient() {}

    EmergencyPatient(string name,int id,string dname,

        float emergency)

        : Patient(name,id,dname)

    {

        emergencyCharge = emergency;

    }

    void calculateBill()

    {

        displayPatient();

        cout << "Bill = "

            << emergencyCharge

```

```

        << endl;

    }

};

//===== MULTIPLE INHERITANCE =====

class CriticalPatient :

    public InPatient,

    public EmergencyPatient
{
public:

    CriticalPatient(string name,

                    int id,

                    string dname,

                    float room,

                    float treatment,

                    float emergency)

: Patient(name,id,dname),

  InPatient(name,id,dname,

            room,treatment),

  EmergencyPatient(name,id,dname,

                  emergency)

{

}

void calculateBill()

{

    displayPatient();

```

```

float total =

    roomCharge +

    treatmentCost +

    emergencyCharge;

cout << "Total Bill = Rs."

    << total

    << endl;

}

};

//===== MAIN =====

int main()

{

    Patient *p = NULL;

    int choice;

    do

    {

        cout << "\n===== HOSPITAL MANAGEMENT =====";

        cout << "\n1. InPatient";

        cout << "\n2. OutPatient";

        cout << "\n3. Emergency Patient";

        cout << "\n4. Critical Patient";

        cout << "\n5. Exit";

        cout << "\nEnter Choice : ";

        cin >> choice;

        string name,doctor;

```

```
int id;

if(choice>=1 && choice<=4)
{
    cout << "Enter Patient Name : ";

    cin >> name;

    cout << "Enter Patient ID : ";

    cin >> id;

    cout << "Enter Doctor Name : ";

    cin >> doctor;
}

switch(choice)
{
case 1:
{
    float room,treatment;

    cout << "Room Charge : ";

    cin >> room;

    cout << "Treatment Cost : ";

    cin >> treatment;

    // Pointer to Derived Class

    InPatient *ip =

    new InPatient(name,id,doctor,

        room,treatment);

    p = ip;
```

```
        break;
    }

    case 2:
    {
        float fee;

        cout << "Consultation Fee : ";

        cin >> fee;

        p = new OutPatient(
            name,id,doctor,fee);

        break;
    }

    case 3:
    {
        float emergency;

        cout << "Emergency Charge : ";

        cin >> emergency;

        p = new EmergencyPatient(
            name,id,doctor,emergency);

        break;
    }

    case 4:
    {

        float room,treatment,emergency;

        cout << "Room Charge : ";
```

```
cin >> room;

cout << "Treatment Cost : ";

cin >> treatment;

cout << "Emergency Charge : ";

cin >> emergency;

// Pointer to Derived Class

CriticalPatient *cp =

new CriticalPatient(

    name,id,doctor,

    room,treatment,

    emergency);

p = cp;

break;

}

case 5:

    cout << "\nProgram Ended";

    break;

default:

    cout << "\nInvalid Choice";

}

if(choice>=1 && choice<=4)

{

    // Runtime Polymorphism

    p->calculateBill();

    delete p;
```

```
    }  
    } while(choice!=5);  
    return 0;  
}
```

Input :

4

Ravi

101

Kumar

2000

3000

1500

Output:

Output
Patient Name : Ravi Patient ID : 101 Doctor : Kumar Total Bill = Rs.6500 === Code Execution Successful ===